GCODECrunch — Development & Performance Report

Project: GCODECrunch G-code travel optimiser **Test set:** `Aircraft/Planks/Gruel/Gruel EMX07/GCODE` **Authorship:** OpenCode & Big Pickle and Prof. Greg Egan (2026) / need prompted by David Benner **Report status:** running document — appended after each experiment iteration.

---

1. Executive summary

GCODECrunch reorders extrusion segments within each print layer to cut non-extruding travel. The work has progressed through three tool generations, six ordering strategies, and an inter-layer coupling pass. This report tracks each step and its measured effect on travel distance across a fixed test set of eight production wing/plank G-code files.

Across the eight files (3 246 703 mm of baseline travel), the optimiser currently removes **63.6 %** of non-extruding travel:

Strategy	Total travel (mm)	Saved vs baseline	Optimizer wall time (s)
baseline (raw source)	3 246 702.7	—	—
`verbatim`	3 047 523.3	6.1 %	18.7
`greedy`	1 365 715.1	57.9 %	19.2
`greedy+2opt` (coupled)	1 197 764.2	63.1 %	56.7
`greedy+2opt+oropt` (uncoupled)	1 213 959.5	62.6 %	30.5
`greedy+2opt+oropt` (coupled)	1 182 242.9	63.6 %	61.7
`auto` (adopt best)	1 182 242.9	63.6 %	163.6

The extruding distance is preserved to within < 0.001 % (correctness invariant). Wall times are for the full eight-file set on the current machine (optimizer runtime, not print time). Per-strategy/per-file optimizer wall times are in §6, Iteration 6.

Three findings stand out. First, the greedy seed already captures most of the available saving; 2-opt and Or-opt add successive ~3 % and ~0.9 %. Second, inter-layer coupling — steering each layer's exit toward the next layer's entry — adds a further **−2.6 %** on top of Or-opt (and is worth even more to plain 2-opt: `greedy+2opt` coupled beats uncoupled `greedy+2opt+oropt`). Third, work is approaching completion: over 82 % of the remaining travel already sits in layers that are fully reorderable, so further gains must come from better solutions to the same per-layer problem (or below whole-segment granularity), not from reordering more layers.

A subtle bug was found and fixed during this work: Or-opt's insertion-cost model used the segment's source orientation rather than its current orientation, producing a handful of moves that *increased* travel (the bug is documented in §6, Iteration 3).

---

2. Objective

1. Reduce non-extruding (travel) distance in sliced G-code without changing the deposited geometry.
2. Keep reversal (boustrophedon) available at all times.
3. Make the segment-ordering strategy selectable, including an automatic "adopt the best solution" mode.
4. Keep right-hand and left-hand wing parts symmetric — mirrored pairs must end up with equal travel.

---

3. Test set and method

3.1 Files

Top-level `*.gcode` in the test folder (sub-folders `OPT/` and `Retired/` excluded):

File	Side	Pair key
`Elevon Horn x 2.gcode`	(single)	—
`Gruel Wing Plates.gcode`	(single)	—
`LOuterGruel VI.gcode` / `ROuterGruel VI.gcode`	L / R	`OuterGruel VI`

File	Side	Pair key
`LR Gruel VI.gcode` / `RR Gruel VI.gcode`	L / R	`R Gruel VI`
`LT Gruel VI.gcode` / `RT Gruel VI.gcode`	L / R	`T Gruel VI`

The `L`/`R` prefix denotes left/right hand; a mirrored pair is expected to exhibit identical travel statistics.

3.2 Metric

`travel_dist` from `gc_engine.analyze_path` / `analyze_lines`: total length of non-extruding XY/Z moves, in millimetres, parsed with the same state machine the optimiser uses. Lower is better. `total_dist` and extruding distance are recorded for context. The extruding distance must be invariant across strategies (a correctness invariant).

3.3 Harness

`scripts/bench_gcode.py` runs the baseline plus each requested strategy over every test file and emits:

- a Markdown results block (pasted here verbatim), and
- `bench_results.csv` (raw per-file/per-strategy metrics).

To refresh the report's own numbers, use `scripts/refresh_report_times.py` instead — it runs the benchmark, rewrites `bench_results.csv`, and replaces the generated block in §6 (the region between the `<!-- BEGIN ... -->` / `<!-- END ... -->` markers) before re-rendering the PDF:

```
python3 scripts/refresh_report_times.py          # ~3 min, ends by writing the PDF
python3 scripts/refresh_report_times.py --dry-run
python3 scripts/refresh_report_times.py --from-csv --no-pdf
```

Nothing inside those markers is hand-maintained, and the generated block carries a *"Measured <date>"* stamp, so the tables cannot silently drift from the data. Only the prose outside the markers is written by hand.

The report deliberately does **not** tabulate estimated print times. `gc_engine.estimate_time` still exists and the benchmark still emits an `est_time_s` column, but the figures are not reliable enough to publish: a feedrate-only model that ignores acceleration, jerk and slicer embellishments disagreed with the slicers' own estimates by tens of percent on this test set, ranging from +50 % to −31 %

depending on how the file interleaves its `F` words with moves. Travel distance is the trustworthy optimisation metric, so that is what the tables report.

---

4. Track of work

Point	Artifact	What changed
v1	`GCODECrunch_v1.py`	First self-contained GUI: folder/file select, NN reorder, temp gradient.
v2	`GCODECrunch_v2.py`	Refactor of the NN GUI (input validation, progress, safer I/O).
v3	`GCODECrunch.py` + `gc_engine.py`	Engine split out as a clean library/CLI; GUI is a thin wrapper.
v3.1	+ temperature gradient	Layer-count-based LW gradient; `enable` checkbox; `M104`/`M109` rewrite only.
v3.2	+ output naming	`{timestamp}_{safe_name}.gcode`; versioned copies; OctoPrint-friendly.
v3.3	+ pluggable orderings	`order` option: `verbatim` / `greedy` / `greedy+2opt` / `auto`.
v3.4	+ 2-opt & auto	Open-path 2-opt local search; `auto` scores candidates and adopts the best.
v3.5	+ Or-opt	Single-segment relocation local search (optionally reversed) run after 2-opt; added to `auto`'s candidate pool.

Point	Artifact	What changed
v3.6	+ inter-layer coupling	`couple_layers` option: re-sweep each layer so its exit steers toward the next layer's entry. Best sweep kept; strictly non-worsening.

---

5. Strategy catalogue

Strategy	Description	Cost
`verbatim`	Emit segments in source order (no reordering). Control case; still re-emits connectors.	O(n)
`greedy`	Greedy nearest-neighbour from the current printer position, optional reversal. The classic v1/v2 behaviour.	O(n^2)/layer
`greedy+2opt`	Greedy seed followed by open-path 2-opt local search (segment count ≤ `two_opt_max_segments`).	O(n^2)–O(n^3)/layer
`greedy+2opt+oropt`	As above, then Or-opt single-segment relocation (`or_opt_max_segments`, `or_opt_max_moves`).	O(n^2)–O(n^3)/layer

Strategy	Description	Cost
`auto`	Builds `greedy`, `greedy+2opt` and `greedy+2opt+oropt`, scores all with `analyze_lines`, adopts the lowest travel. Reports which won.	sum of all

Baseline (raw source file, no re-emit) is also reported for reference. `verbatim` differs from baseline only because connectors are regenerated on re-emit.

5.1 Inter-layer coupling (`couple_layers`)

Orthogonal to the ordering strategy. When `couple_layers ≥ 1`, `_nn_build` runs a first pass, records each layer's chosen entry point, then re-sweeps the whole file several times passing each layer a target: the entry point of the following layer. The local searches then optimise the path as *start* → … → *target*, so a layer is encouraged to finish close to where the next one begins. The connector between layers is the regenerated travel into the next layer's first segment, whose length is exactly `distance(exit_prev, entry_next)`.

Because `_nn_build` scores every sweep with `analyze_lines` and keeps the lowest, a non-zero `couple_layers` can only improve or equal the travel; sweeps are separate from the ordering strategy. Cost is roughly `(1 + couple_layers)` × the uncoupled time. Default is 1.

---

6. Experiment log

Iteration 1 — establish the baseline and per-strategy envelope

Hypothesis: `greedy` improves materially over baseline; `greedy+2opt` improves further on layers small enough to enter the 2-opt pass; `auto` never loses to `greedy` and equals `greedy+2opt` where it helps.

Results (travel mm; wall time is per-file, full-set totals in §1):

File	Lines	Layers	Segs	Baseline	verbatim	greedy	greedy+2opt	auto
Elevon 6	245	16	174	3 000.4	2 772.0	2 350.9	2 253.9	2 253.9
Horn x 2	22	25	852	9	7	4	4	4
Gruel	730			914.5	347.9	550.5	196.2	196.2
Wing Plates	124	488	3 813	154	141	57	56	56
LOuterGruel VI	759			557.9	374.4	185.5	936.6	936.6
LR	562	520	13	443	412	176	145	145
Gruel VI	531		705	449.0	196.7	634.3	831.1	831.1
LT	1 041	2 206	22	871	815	450	411	411
Gruel VI	852		067	749.7	256.3	211.2	455.6	455.6
ROuterGruel VI	120 928	488	3 810	153 932.7	141 362.2	57 533.6	56 966.4	56 966.4
RR	629	612	15	562	530	162	139	139
Gruel VI	242		719	574.2	064.0	974.4	340.5	340.5
RT	1 039	2 206	22	1 047	997	454	408	408
Gruel VI	161		069	524.3	149.8	274.7	271.2	271.2

Totals: baseline 3 246 702.7; `verbatim` 3 047 523.3 (−6.13 %); `greedy` 1 365 715.1 (−57.94 %); `greedy+2opt` 1 225 251.4 (−62.26 %); `auto` equal to `greedy+2opt` (−62.26 %). 2-opt beat greedy by a further 10.3 %.

The extruding-distance invariant held: deltas were < 0.001 % (largest ≈ +2.58 mm over a ~326 000 mm extrusion total).

Commentary. The dominant win is the greedy nearest-neighbour seed; 2-opt adds a solid second helping. `auto` chose `greedy+2opt` on every file — as expected, since 2-opt never makes a layer worse. This iteration left the question: how much travel remains, and can a stronger local search reach it?

Iteration 2 — is the optimiser symmetric under mirroring?

Hypothesis: if the segment-ordering algorithm is mirror-invariant, then any left/right travel difference is inherited from the input files, not introduced by the optimiser.

Method. Take `LT Gruel VI`, mirror it about its own mid-plane (X span 0…967, k = 967), and run every strategy on both. A mirror-invariant

optimiser yields a constant offset between original and mirror for every strategy.

Results (travel mm):

Strategy	original	mirrored	delta
raw baseline	871 749.7	872 690.8	−941.1
`verbatim`	815 256.3	816 197.4	−941.1
`greedy`	450 211.2	451 152.4	−941.1
`greedy+2opt`	411 455.6	412 396.7	−941.1

The delta is **identical (−941.1 mm) for all strategies.**

Commentary. The optimiser adds *zero* asymmetry: mirroring the input moves the result by exactly the same amount the raw file moves. Therefore any residual L−R travel gap is a property of the source geometry, not of the reordering. This redirected the symmetry investigation to comparing the actual input files.

Iteration 3 — Or-opt (single-segment relocation)

Motivation. A per-layer travel breakdown showed the remaining travel is not in unreachable corners: it sits in layers that *are* being reordered.

File	start-up block	reordered layers (≥2 segs)	verbatim layers (<2 segs)	reordered layers
LT Gruel VI	27 mm	338 666 mm	72 763 mm	2 206
RR Gruel VI	27 mm	139 313 mm	0 mm	612
LOuterGruel VI	27 mm	56 910 mm	0 mm	488

For `RR` and `LOuter`, **100 %** of remaining travel is inside reorderable layers; for `LT`, 82 %. So a stronger per-layer search had a clear target. Segment counts per layer are small — the maximum anywhere in the test set is 366, and the vast majority of layers hold 3–40 segments — so an additional $O(n^2)$ local search is affordable.

Implementation. `or_opt` pulls one segment out of the ordering and re-inserts it at the cheapest gap, in either orientation. It runs after 2-opt and is guarded by `or_opt_max_segments` (default 120)

and `or_opt_max_moves` (default 150). It was added to `ORDERINGS` as `greedy+2opt+oropt` and to the `auto` candidate pool.

Bug found and fixed. The first run *increased* travel on `LT` (+1 638 mm) and `RT` (+1 105 mm) — impossible for a correct descent. Instrumenting a single layer isolated the fault: the insertion cost was computed as if inserting the segment in its *source* orientation (`rev == False` ⇒ entry = `seg.start`). When the segment was *currently reversed* in the ordering (`rev_i == True`), its current entry was actually `seg.end`, so the predicted entry/exit were swapped and the move was scored as an improvement when it was in fact a worsening. The fix is a one-token change — keep orientation relative to the segment's current direction:

```
c, f = (a_i, b_i) if rev == rev_i else (b_i, a_i)
```

After the fix the per-layer validator (`path_cost` before/after Or-opt) reported **0 worsening layers** and a net within-layer gain of 46 828 mm on `LT`. `two_opt` was audited too; it uses precomputed boundary arrays and is orientation-agnostic, so it was already correct.

Results (travel mm; full-set totals in §1):

File	Baseline	greedy+2opt	greedy+2opt+oropt	greedy+2opt+oropt	Δ vs 2opt
Elevon Horn x 2	3 000.4	2 253.9	1 847.6	1 847.6	−18.0 %
Gruel Wing Plates	9 914.5	4 196.2	4 084.6	4 084.6	−2.7 %
LOuterGruel VI	154 557.9	56 936.6	56 958.2	56 958.2	+0.04 %
LR	443	145	142	142	−2.0 %
Gruel VI	449.0	831.1	877.9	877.9	
LT	871	411	411	411	−0.10 %
Gruel VI	749.7	455.6	059.5	059.5	
ROuterGruel VI	153 932.7	56 966.4	56 967.0	56 967.0	+0.001 %
RR	562	139	132	132	−5.2 %
Gruel VI	574.2	340.5	101.7	101.7	
RT	1 047	408	408	408	−0.05 %
Gruel VI	524.3	271.2	063.0	063.0	

Totals: `greedy+2opt+oropt` 1 213 959.5 (−62.61 % vs baseline), a further 11 292 mm (0.92 %) below 2-opt. `auto` scored 1 213 937.3 and selected Or-opt on six files, falling back to plain 2-opt on the two `Outer` files where Or-opt was a hair worse (auto's whole-file scoring catches this automatically).

Commentary. Or-opt is a clear win on layers with more segments (Elevon 18 %, RR 5.2 %, LR 2.0 %) and near-neutral on simple layers, where 2-opt already reaches the optimum. The time cost is modest (+3.1 s over the whole set). The larger lesson is the bug: a local search's move-evaluation must be proven non-worsening by an independent before/after check before its numbers are trusted — the first "improvement" numbers looked plausible yet were wrong.

Iteration 4 — remaining-travel accounting

After Or-opt, the same breakdown shows how little is left outside reachable layers (82–100 % of residual travel is in reordered layers), and the per-layer validator shows 2-opt+Or-opt is at/near the local optimum for typical 3–40-segment layers. The residual is therefore structural: whole-segment granularity and per-layer isolation (see §8), not a missing reorder opportunity.

Status: complete.

Iteration 5 — inter-layer coupling

Motivation. Every layer was ordered in isolation: the search minimised the path from the layer's entry but treated the layer's *exit* as free. Yet the connector into the next layer costs `distance(exit_prev, entry_next)`, so a layer that ends far from where the next one begins pays for it. The presentation idea that first suggested a fixed target turned out not to apply — `emit_segment` strips the prologue's XY travels and regenerates them — so both `entry_next` and `exit_prev` are free choices, and the objective is a chain. That calls for an alternating sweep rather than a one-pass lookahead.

Implementation. `two_opt` and `or_opt` gained an optional `end_target`; the path is then scored as *start* → … → *target*, and Or-opt is free to decide which segment finishes the layer. `_nn_build` was rebuilt around *sweeps*:

1. parse every chunk into units (fixed blocks vs reorderable layers) — untouched;
2. sweep 0 orders each layer with no target (the old behaviour);
3. each later sweep re-emits the file, passing every layer the *previous* sweep's entry point of the next layer as its target;

4. every sweep is scored with `analyze_lines` and the best text is kept.

Because sweep 0 is a candidate, coupling is **strictly non-worsening**. It is gated to the local-search orderings (`greedy+2opt`, `greedy+2opt+oropt`); `greedy` and `verbatim` have no exit-steering move, so coupling is skipped for them. Default `couple_layers = 1`.

Convergence check (travel mm, `greedy+2opt+oropt`):

File	couple=0	couple=1	couple=2	couple=3
LT Gruel VI	411 059.5	391 118.9	391 118.9	391 118.9
RT Gruel VI	408 063.0	396 972.0	396 972.0	396 972.0
RR Gruel VI	132 101.7	131 614.7	131 516.5	131 516.5

One sweep captures essentially all the gain; additional sweeps are near-neutral (`RR` gains a further 0.07 % at sweep 2). The extruding invariant held to < 0.006 mm.

Results. Full per-file comparison (`@0` = uncoupled):

File	Baseline	greedy+2opt	oropt @0	oropt coupled	Δ coupled vs @0
Elevon Horn x 2	3 000.4	2 253.9	1 847.6	1 847.6	0
Gruel Wing Plates	9 914.5	4 196.2	4 084.6	4 084.6	0
LOuterGruel VI	154 557.9	56 879.9	56 958.2	56 852.1	−106.1
LR Gruel VI	443 449.0	145 831.1	142 877.9	142 877.9	0
LT Gruel VI	871 749.7	393 683.9	411 059.5	391 118.9	−19 940.6
ROuterGruel VI	153 932.7	56 883.5	56 967.0	56 875.1	−91.9
RR Gruel VI	562 574.2	138 933.6	132 101.7	131 614.7	−487.0

File	Baseline	greedy+2opt	oropt @0	oropt coupled	Δ coupled vs @0
RT	1 047	399	408	396	−11
Gruel VI	524.3	102.1	063.0	972.0	091.0

Totals: uncoupled `greedy+2opt+oropt` 1 213 959.5 (−62.61 %); coupled 1 182 242.9 (−63.59 %), a further **31 716.6 mm (−2.61 %)** removed. Coupling also lifts `greedy+2opt` to **1 197 764.2** (−63.11 %) — i.e. coupled 2-opt now beats uncoupled 2-opt+Or-opt.

Commentary. Coupling is the largest single addition since the greedy seed and the first to exploit structure *between* layers. It is cheap to reason about (non-worsening, best sweep kept) and, pleasingly, helps the weaker strategy (`greedy+2opt`) proportionally more, because that strategy benefits most from a well-chosen final edge. The two large `T` files dominate the gain; the dense `LR` and the small `Elevon/Wing` files were already well coupled, and `auto` selects the coupled result everywhere it helps.

Iteration 6 — Optimizer wall time

This is the *compute* time the reorder/search passes take to produce each optimised file, measured with `time.perf_counter()` around each strategy run in `bench_gcode.py` (`seconds` column in `bench_results.csv`).

Optimizer wall time per file (seconds), `greedy+2opt+oropt` (coupled):

File	Lines	Segments	Optimizer time (s)	Time per 1k lines (ms)
Elevon Horn x 2.gcode	6 245	174	0.11	17.5
Gruel Wing Plates.gcode	22 730	852	1.55	68.1
LOuterGruel VI.gcode	121 759	3 813	1.83	15.0
LR Gruel VI.gcode	562 531	13 705	13.42	23.9

File	Lines	Segments	Optimizer time (s)	Time per 1k lines (ms)
LT Gruel VI.gcode	1 041 852	22 067	14.97	14.4
ROuterGruel VI.gcode	120 928	3 810	1.66	13.7
RR Gruel VI.gcode	629 242	15 719	13.36	21.2
RT Gruel VI.gcode	1 039 161	22 069	17.03	16.4
TOTAL	3 544 448	82 209	**63.91**	18.0

Optimizer wall time per strategy (full eight-file set, seconds):

Strategy	Optimizer wall time (s)	Travel saved	s per % travel saved
`verbatim`	18.7	6.1 %	3.07
`greedy`	19.2	57.9 %	0.33
`greedy+2opt` (coupled)	56.7	63.1 %	0.90
`greedy+2opt+oropt` (uncoupled)	30.5	62.6 %	0.49
`greedy+2opt+oropt` (coupled)	61.7	63.6 %	0.97
`auto` (adopt best)	163.6	63.6 %	2.57

Commentary. Cost tracks *segments per layer*, not line count, because the local searches are $O(n^2)$–$O(n^3)$ per layer. `Gruel Wing Plates` packs its segments into a couple of dozen very dense layers, so it costs far more per thousand lines than `LT`, which spreads a much larger segment budget over ~2 200 thin layers and is ~45× longer yet cheaper to optimise end to end. `greedy` is by far the most efficient strategy — it captures the bulk of the saving for seconds of work — and the local searches spend a few times more to buy only the last handful of points. `auto` is the most expensive, because it builds and scores every candidate before adopting the best.

All of these are trivially cheap next to a print lasting tens of hours: the generated figures above put the return on compute at roughly two orders of magnitude. Timings vary by a few percent between runs, and the per-strategy totals quoted in §1 are a separate measurement, so treat small differences between the two as run-to-run noise.

Iteration 7 — (next)

Planned: segment splitting and/or a cheapest-insertion seed (§9).

---

7. R/L symmetry analysis

7.1 Optimiser is mirror-invariant

Proven in Iteration 2: mirroring the input shifts every strategy's travel by the same constant. The optimiser contributes no asymmetry.

7.2 Are the input pairs actually mirrored?

An extruding bounding-box / Z-range scan of each pair:

Pair	width L/R	Y-height L/R	Z-range L/R	points L/R	Mirrored?
OuterGruel VI	160.193 / 160.192	238.798 / 238.798	0.20–96.60	56 268 / 56 487	yes
T Gruel VI	160.264 / 160.263	246.677 / 246.677	0.20–324.20	458 104 / 457 184	yes
R Gruel VI	158.941 / 158.629	249.887 / 249.772	0.20–97.40 / 0.20–115.80	279 149 / 312 267	**no**

`LR/RR` differ not only in point count but in height (Z range 97.40 vs 115.80) and layer count (520 vs 612) — they are **not** mirrored parts. `Outer` and `T` are close mirrors but not bit-identical (point counts differ by ~200–900, likely slicer tie-breaks and per-layer start-point conventions).

7.3 Measured L−R travel difference (mm; L minus R)

Pair	Baseline	verbatim	greedy	greedy+2opt	oropt @0	oropt coupled
OuterGruel VI	+625.2	+12.2	−348.1	−3.7	−8.8	−23.0
R Gruel VI	−119 125.2	−117 867.2	+13 659.9	+6 897.5	+10 776.2	+11 263.2

Pair	Baseline	verbatim	greedy	greedy+2opt	oropt @0	oropt coupled
T	−175	−181	−4	−5	+2	−5
Gruel VI	774.6	893.5	063.5	418.2	996.5	853.1

Commentary. For the true mirrors (`Outer`, `T`) the optimiser collapses the gap from hundreds of thousands of mm (baseline) to a few thousand mm, and coupling takes `Outer` to −23.0 mm. The `T` figure moves with coupling (−5 853 mm; ~1.5 % of one file's travel) because the two inputs are close but not bit-identical mirrors, so the coupled sweep settles into slightly different local optima on each side. For `R Gruel` no strategy can equalise the pair, because the inputs are not mirrors (§7.2) — the optimiser is faithfully optimising two different shapes. The conclusion requested at project start — "L/R pairs should end up equal" — holds to the extent the inputs are mirrored; where they are not, the residual is a data issue, not an algorithm issue.

---

8. Known remaining-travel causes

Measured after Or-opt + coupling. Over 82 % of the residual travel still sits in layers that *are* reordered, and the per-layer local search is at/near optimum for typical layers, so the remaining causes are structural:

- **Whole-segment granularity** — a long segment is treated as one unit; no splitting at an interior point to shorten a connector.
- **Mandatory non-XY moves** — Z lifts, retracts, and the start-up/trailing blocks (27 mm start-up is never reordered; layers with < 2 segments are verbatim).
- **Greedy seed ceiling** — the seed is typically 15–30 % above optimal, and the local searches only partially close that gap on large layers.
- **Large-layer guard** — layers above `or_opt_max_segments` (120) and `two_opt_max_segments` (800) skip the corresponding search; only a handful of layers in this set exceed them.
- **Coupling convergence** — one sweep captures essentially all the gain; a second sweep buys <= 0.1 %. The chain is solved to a good, not provably global, optimum.

Per-layer isolation was the largest listed cause in earlier drafts; it is now addressed by `couple_layers` (Iteration 5).

---

9. Next steps

1. **Segment splitting** — allow a long segment to be split to shorten a connector (changes deposited geometry timing, so needs care). The headline remaining lever.
2. **Better seed** — cheapest-insertion or nearest-edge seed to lift the large-layer ceiling.
3. **Coupling follow-ups** — target the *entry* as well as the exit (currently only the exit is steered), or alternate forward/backward sweeps.
4. **Or-opt follow-ups** (cheap, likely small) — raise `or_opt_max_segments` or run a second Or-opt pass after a fresh 2-opt sweep.
5. Re-render this Markdown to PDF for the consolidated deliverable.